Assignment 4 Report

Maryna De Almeida Brito

March 31, 2026

Course: CMPT276 D300

1 Code Smells Identified

The teammate whose code I reviewed was Komal Kaur. I reviewed the classes GameView, HomePage, WinningView and LosingByHearts, finding total of four code smells. Most of the smells involved unused or confusing variable names, dead code and classes that were too large. The HomePage, LosingByHearts and WinningView classes shared the same code smell, variable Game g was being set in the constructor, but never used, rendering it dead code. The HomePage and LosingByHearts classes had two variables that shared the same name, private Game g set in the constructor and the parameter in paintComponent(Graphics g) method. The variable g makes poor readability and confusion when being called by other methods as there's no way determine which one is being used, just through inspection. Furthermore, the getter methods in LosingByHearts class, such as getBX, getBY, BWidth and BHeight were ambiguous as the letter "B" could represent either a button or a box. GameView class contained the most significant code smell regarding class size. The purpose of the class was to handle the visuals of the game world (barriers, enemies, rewards, etc.), but it was also dealing with the tutorial view. As like other classes created in the game.ui package, tutorial view should have its own class as it diverges from the goal of GameView and makes the class larger, dealing with too many things at once. I also provided other comments on removing the imports not being used and adding more detail to JavaDoc descriptions.

2 Steps and Re-factoring

The team-mate who reviewed my code was Nathaniel Chen. He reviewed the MovingEnemy, Animal, Pirate and Explorer classes, finding six code smells related to confusing variable names, unnecessary if/else statement, reduced encapsulation and poorly structured code. For the Explorer class the steps for re-factoring were straightforward. The method deactivateShield had a poor structure and wasn't doing its proper work. The method is supposed to deactivate the shield if the user hits a moving or static enemy. The shield time is supposed to be set back to 0 and boolean hasShield to false, but before I was addressing one of the two steps required. I re-factored the code by adding shieldTicksRemaining = 0 as it wasn't set before. Another method issue was with the collectreward method having an unnecessary else if statement, I was also checking if reward was an instance of coin and updating the score inside the else if statement by calling game.updateScore(reward.getValue()), but that's not necessary as the value of shield is 0, since shield doesn't affect the score. So, I removed the else if statement and just called game.updateScore(reward.getValue()) right after the if statement before reward.collect(). The last code smell in the Explorer class was about package private variable, the variable Game game didn't have any modifier, I changed to private Game game as that variable was being used by other public methods like setGame. The MovingEnemy class had simple and complex smells. There was a variable called "defaulPosition" which had a typo because it was supposed to be "default" instead of "defaul", so I

changed the variable name to "defaultPosition". There was also a similar code smell to the Explorer class as the variables "defaultPosition", "random" and "directions" were all package private, so I re-factored them all to private. However, when I set "defaultPosition" to private, the variable had no other references beyond being set in the constructor leading to a dead code, so I removed it. The most complex code smell in the MovingEnemy class was the smell on move() method. The method was modifying pixelX and pixelY without checking if those modifications were valid. I resolved this by adding new potential coordinate variables, checking if they were valid and setting them to the pixel variables. First I created potentialX and potentialY variables and set them equal to pixelX + velocityX and pixelY + velocityY, doing the same thing as the old code, but addressing with a new variable. Inside the gridX and gridY variables I called potentialX / tileSize and potentialY / tileSize and renamed newGridPos to potentialGridPos as it makes more sense since its checking if the position is valid or not. If the if statement was true I set it pixelX += velocityX and pixelY += velocityY inside it. However, when I went to test if the game was working, the moving enemies were going over the hedges and trees. After reviewing the re-factored code, I realized that in the else statement the part setting pixelX -= velocityX and pixelY -= velocityY, for the enemy to 'step back' if it hit a hedge or a tree was no longer needed as pixels were only being changed if the position was valid. Therefore, I removed those two lines of code and just left the else statement to reset velocities to 0. My team-mate also made other comments about JavaDoc documentation, so I added the author and version comments that were missing, along with comments for getVelocityX and getVelocityY in the MovingEnemy class. The Animal and Pirate classes didn't have any code smells.